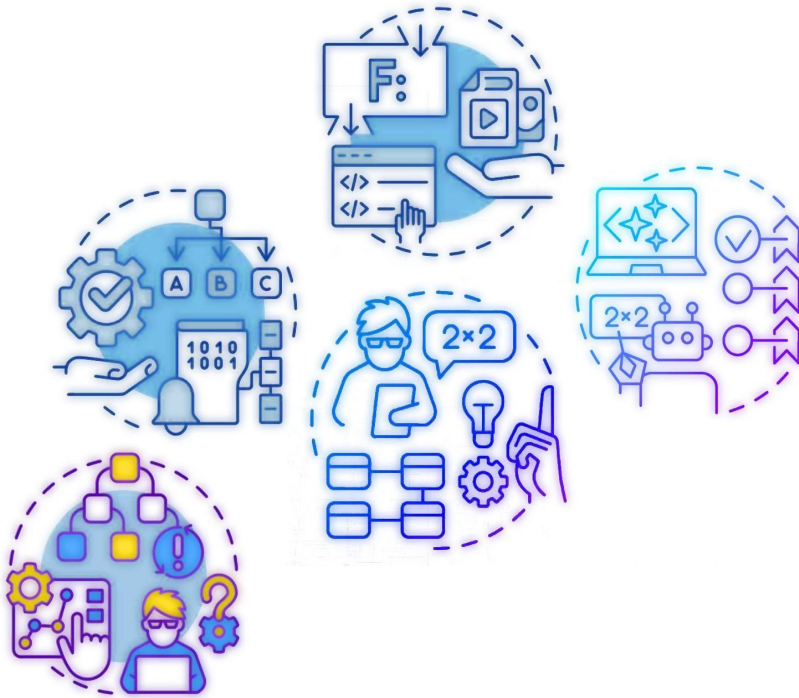




Paradigma Orientado a Objetos

Esp. Ing. Viviana A. Santucci
AIA Federico Castoldi
Ing. V. Franco Matzkin
Ing. Cecilia Serafini

Tecnologías de la Programación

1 – Patrones

Los patrones de diseño (design patterns) son la base para la búsqueda de soluciones a problemas comunes en el desarrollo de software y otros ámbitos referentes al diseño de interacción o interfaces.

Objetivos

- Evitar la reiteración en la búsqueda de soluciones a problemas ya conocidos y solucionados anteriormente.
- Formalizar un vocabulario común entre diseñadores.
- Estandarizar el modo en que se realiza el diseño.
- Facilitar el aprendizaje de las nuevas generaciones de diseñadores condensando conocimiento ya existente.

1 – Patrones

No es obligatorio utilizar los patrones, solo es aconsejable en el caso de tener el mismo problema o similar que soluciona el patrón, siempre teniendo en cuenta que en un caso particular puede no ser aplicable.

Abusar o forzar el uso de los patrones puede ser un error

1 – Patrones

Un patrón de diseño nomina, abstrae e identifica los aspectos clave de una estructura de diseño común, lo que los hace útiles para crear un diseño orientado a objetos reutilizable. El patrón de diseño identifica las clases e instancias participantes, sus roles y colaboraciones, y la distribución de responsabilidades. Cada patrón de diseño se centra en un problema concreto, describiendo cuándo aplicarlo y si tiene sentido hacerlo teniendo en cuenta otras restricciones de diseño, así como las consecuencias, ventajas e inconvenientes de su uso.

Los patrones de diseño proveen una forma efectiva para compartir experiencia con la comunidad de programadores de software orientado a objetos.

1.1 – Componentes

Para que una solución sea considerada un patrón debe poseer ciertos componentes fundamentales:

- Nombre del patrón. Permite describir en pocas palabras un problema de diseño junto con sus soluciones y consecuencias.
- Problema (o fuerzas no balanceadas). Indica cuándo aplicar el patrón. En algunas oportunidades el problema incluye una serie de condiciones que deben darse para aplicar el patrón. Muestra la verdadera esencia del problema. Este enunciado se completa con un conjunto de fuerzas, término que se utiliza para indicar cualquier aspecto del problema que deba ser considerado a la hora de resolverlo, entre ellos:
 - Requerimientos a cumplir por la solución.
 - Restricciones a considerar.
 - Propiedades deseables que la solución debe tener.

1.1 – Componentes

- Solución. No describe una solución o implementación en concreto, sino que un patrón es más bien como una plantilla que puede aplicarse en diversas situaciones diferentes. El patrón brinda una descripción abstracta de un problema de diseño y cómo lo resuelve una determinada disposición de objetos. Que la solución sea aplicable a diversas situaciones denota el carácter “recurrente” de los patrones.
- Consecuencias. Resultados en términos de ventajas e inconvenientes.

1.2 – Tipos

Según el nivel de abstracción los patrones de diseño pueden clasificarse de la siguiente forma:

- Patrones arquitectónicos. Centrados en la arquitectura del sistema. Definen una estructura fundamental sobre la organización del sistema. Proveen un conjunto predefinido de subsistemas, cuáles son sus responsabilidades y como se interrelacionan.
- Patrones de diseño. Esquemas para refinar los subsistemas o componentes de un sistema de software, o sus relaciones. Describen una estructura recurrente y común de componentes comunicantes que resuelven un problema de diseño dentro de un contexto. Ejemplo: el patrón singleton asegura que exista sólo una instancia de una determinada clase.
- Patrones de codificación o modismos (idioms). Patrones que ayudan a implementar aspectos particulares del diseño en un lenguaje de programación específico.

1.2 – Tipos

Según el nivel de abstracción los patrones de diseño pueden clasificarse de la siguiente forma:

- Patrones arquitectónicos. Centrados en la arquitectura del sistema. Definen una estructura fundamental sobre la organización del sistema. Proveen un conjunto predefinido de subsistemas, cuáles son sus responsabilidades y como se interrelacionan.
- Patrones de diseño. Esquemas para refinar los subsistemas o componentes de un sistema de software, o sus relaciones. Describen una estructura recurrente y común de componentes comunicantes que resuelven un problema de diseño dentro de un contexto. Ejemplo: el patrón singleton asegura que exista sólo una instancia de una determinada clase.
- Patrones de codificación o modismos (idioms). Patrones que ayudan a implementar aspectos particulares del diseño en un lenguaje de programación específico.

1.3 – Catálogo

Los patrones de diseño refinan subsistemas o componentes de software orientado a objetos. Estos describen una estructura genérica sobre el problema a resolver, de manera tal de poder contextualizarla con el problema a resolver. (Fowler, 2003) sostiene que “están a medio cocer” ya que siempre se los debe contextualizar al entorno en el que serán aplicados. De hecho, una de las mejores formas de aprender sobre ellos es implementarlos uno mismo.

El **principal catálogo de patrones de diseño** es (Gamma et al, 1995), el cual **consta de 23 patrones de diseño** agrupados en tres grupos:

- Patrones Creacionales,
- Patrones Estructurales y
- Patrones de Comportamiento

1.3.1 – Catálogo - Patrones Creacionales

Los patrones creacionales abstraen el proceso de instanciación de objetos, ayudando a que el sistema sea independiente de cómo se crean, componen y representan sus objetos. Estos patrones encapsulan el conocimiento sobre las clases concretas que utiliza el sistema.

En otras palabras, estos patrones brindan soporte a una de las tareas mas comunes dentro de la programación orientada a objetos: la instanciación.

1.3.1 – Catálogo - Patrones Creacionales

Estos patrones brindan las siguientes características:

- Instanciación genérica: permite que los objetos sean creados dentro del sistema sin especificar clases concretas en el código.
- Simplicidad: algunos patrones facilitan la creación de objetos, evitando que el cliente deba tener código complejo sobre como instanciar un determinado objeto.
- Restricciones creacionales: algunos patrones ayudan a establecer restricciones sobre la creación de objetos, tales como qué objeto crear, cuándo, cómo, etc.

1.3.1 – Catálogo - Patrones Creacionales

Los patrones creacionales son los siguientes:

1. **Singleton**
2. Abstract Factory
3. Factory Method
4. Builder
5. Prototype

1.3.1 – Catálogo - Patrones Creacionales

Singleton: <https://refactoring.guru/es/design-patterns/singleton>

Resuelve dos problemas al mismo tiempo, vulnerando el *Principio de responsabilidad única*:

1. Garantizar que una clase tenga una única instancia. ¿Por qué querría alguien controlar cuántas instancias tiene una clase? El motivo más habitual es controlar el acceso a algún recurso compartido, por ejemplo, una base de datos o un archivo.
2. Proporcionar un punto de acceso global a dicha instancia. ¿Recuerdas esas variables globales que utilizaste (bueno, sí, fui yo) para almacenar objetos esenciales? Aunque son muy útiles, también son poco seguras, ya que cualquier código podría sobrescribir el contenido de esas variables y descomponer la aplicación.

1.3.1 – Catálogo - Patrones Creacionales

Singleton: <https://refactoring.guru/es/design-patterns/singleton>

Solución

Todas las implementaciones del patrón Singleton tienen estos dos pasos en común:

- Hacer privado el constructor por defecto para evitar que otros objetos utilicen el operador `new` con la clase Singleton.

- Crear un método de creación estático que actúe como constructor. Tras bambalinas, este método invoca al constructor privado para crear un objeto y lo guarda en un campo estático. Las siguientes llamadas a este método devuelven el objeto almacenado en caché.

1.3.1 – Catálogo - Patrones Creacionales

Singleton: <https://refactoring.guru/es/design-patterns/singleton>

Consecuencias (Ventajas)

- Tener la certeza de que una clase tiene una única instancia.
- Obtener un punto de acceso global a dicha instancia.
- El objeto Singleton solo se inicializa cuando se requiere por primera vez.

1.3.1 – Catálogo - Patrones Creacionales

Singleton: <https://refactoring.guru/es/design-patterns/singleton>

Consecuencias (Desventajas)

- Vulnera el *Principio de responsabilidad única*. El patrón resuelve dos problemas al mismo tiempo.
- Puede enmascarar un mal diseño, por ejemplo, cuando los componentes del programa saben demasiado los unos sobre los otros.
- Requiere de un tratamiento especial en un entorno con múltiples hilos de ejecución, para que varios hilos no creen un objeto Singleton varias veces.
- Puede resultar complicado realizar la prueba unitaria del código cliente del Singleton porque muchos *frameworks* de prueba dependen de la herencia a la hora de crear objetos simulados (mock objects). Debido a que la clase Singleton es privada y en la mayoría de los lenguajes resulta imposible sobrescribir métodos estáticos, tendrás que pensar en una manera original de simular el Singleton. O, simplemente, no escribas las pruebas. O no utilices el patrón Singleton.

1.3.2 – Catálogo - Patrones Estructurales

Se encargan de cómo se combinan clases y objetos para formar estructuras más grandes. Los patrones estructurales de clases utilizan la herencia para componer interfaces o implementaciones.

En lugar de combinar interfaces o implementaciones, los patrones estructurales de objetos describen formas de componer objetos para obtener nuevas funcionalidades.

La flexibilidad añadida mediante la composición de objetos viene dada por la capacidad de cambiar la composición en tiempo de ejecución, que es imposible con la composición de clases.

Ejemplos típicos son cómo comunicar dos clases incompatibles o cómo añadir funcionalidad a objetos.

1.3.2 – Catálogo - Patrones Estructurales

Los patrones estructurales son los siguientes:

1. Adapter
2. Bridge
3. **Composite**
4. Decorator
5. Facade
6. Flyweight
7. Proxy:
 - a. Proxy remoto
 - b. Proxy virtual
 - c. Proxy de protección

1.3.2 – Catálogo - Patrones Estructurales

Composite (<https://refactoring.guru/es/design-patterns/composite>)

Problema: El uso del patrón Composite sólo tiene sentido cuando el modelo central de la aplicación puede representarse en forma de árbol.

Por ejemplo, existen dos tipos de objetos: Productos y Cajas. Una Caja puede contener varios Productos así como cierto número de Cajas más pequeñas. Estas Cajas pequeñas también pueden contener algunos Productos o incluso Cajas más pequeñas, y así sucesivamente.

Digamos que se decide crear un sistema de pedidos que utiliza estas clases. Los pedidos pueden contener productos sencillos sin envolver, así como cajas llenas de productos... y otras cajas. ¿Cómo determinarás el precio total de ese pedido?

1.3.2 – Catálogo - Patrones Estructurales

Composite (<https://refactoring.guru/es/design-patterns/composite>)

Solución

El patrón Composite sugiere trabajar con **Productos** y **Cajas** a través de una interfaz común que declara un método para calcular el precio total.

¿Cómo funcionaría este método? Para un producto, sencillamente devuelve el precio del producto. Para una caja, recorre cada artículo que contiene la caja, pregunta su precio y devuelve un total por la caja. Si uno de esos artículos fuera una caja más pequeña, esa caja también comenzaría a repasar su contenido y así sucesivamente, hasta que se calcule el precio de todos los componentes internos. Una caja podría incluso añadir costos adicionales al precio final, como costos de empaquetado.

1.3.2 – Catálogo - Patrones Estructurales

Composite (<https://refactoring.guru/es/design-patterns/composite>)

Consecuencias (Ventajas)

- Trabajar con estructuras de árbol complejas con mayor comodidad: utiliza el polimorfismo y la recursión a favor del desarrollador.
- *Principio de abierto/cerrado*. Introducir nuevos tipos de elemento en la aplicación sin descomponer el código existente, que ahora funciona con el árbol de objetos.

1.3.2 – Catálogo - Patrones Estructurales

Composite (<https://refactoring.guru/es/design-patterns/composite>)

Consecuencias (Desventajas)

- Puede resultar difícil proporcionar una interfaz común para clases cuya funcionalidad difiere demasiado. En algunos casos, se tiene que generalizar en exceso la interfaz componente, provocando que sea más difícil de comprender.

1.3.3 – Catálogo - Patrones de comportamiento

Tienen que ver con algoritmos y asignación de responsabilidades.

Se focalizan en el flujo de control dentro de un sistema. Ciertas formas de organizar los controles dentro del sistema pueden llevar a grandes beneficios en cuanto a mantenibilidad y eficiencia. Algunos ejemplos de estos patrones incluyen la definición de abstracciones de algoritmos, las colaboraciones entre objetos para realizar tareas complejas reduciendo las dependencias o asociar comportamiento a objetos e invocar su ejecución.

Los patrones de comportamiento basados en clases utilizan la herencia para distribuir el comportamiento entre clases, ellos son: Template Method e Interpreter. Mientras que los basados en objetos utilizan la composición.

1.3.3 – Catálogo - Patrones de comportamiento

Los patrones de comportamiento son los siguientes:

1. Chain of responsibility
2. Command
3. Interpreter
4. Iterator
5. Mediator
6. Memento
7. **Observer**
8. State
9. Strategy
10. Template Method

1.3.3 – Catálogo - Patrones de comportamiento

Observer (<https://refactoring.guru/es/design-patterns/observer>)

Problema

Imagina que tienes dos tipos de objetos: un objeto Cliente y un objeto Tienda. El cliente está muy interesado en una marca particular de producto (digamos, un nuevo modelo de iPhone) que estará disponible en la tienda muy pronto.

El cliente puede visitar la tienda cada día para comprobar la disponibilidad del producto. Pero, mientras el producto está en camino, la mayoría de estos viajes serán en vano.

1.3.3 – Catálogo - Patrones de comportamiento

Observer (<https://refactoring.guru/es/design-patterns/observer>)

Problema

Por otro lado, la tienda podría enviar cientos de correos (lo cual se podría considerar spam) a todos los clientes cada vez que hay un nuevo producto disponible. Esto ahorraría a los clientes los interminables viajes a la tienda, pero, al mismo tiempo, molestaría a otros clientes que no están interesados en los nuevos productos.

Parece que nos encontramos ante un conflicto. O el cliente pierde tiempo comprobando la disponibilidad del producto, o bien la tienda desperdicia recursos notificando a los clientes equivocados.

1.3.3 – Catálogo - Patrones de comportamiento

Observer (<https://refactoring.guru/es/design-patterns/observer>)

Solución

El objeto que tiene un estado interesante suele denominarse *sujeto*, pero, como también va a notificar a otros objetos los cambios en su estado, le llamaremos *notificador* (en ocasiones también llamado *publicador*). El resto de los objetos que quieren conocer los cambios en el estado del notificador, se denominan *suscriptores*.

El patrón Observer sugiere que añadas un mecanismo de suscripción a la clase notificadora para que los objetos individuales puedan suscribirse o cancelar su suscripción a un flujo de eventos que proviene de esa notificadora. ¡No temas! No es tan complicado como parece. En realidad, este mecanismo consiste en: 1) un campo matriz para almacenar una lista de referencias a objetos suscriptores y 2) varios métodos públicos que permiten añadir suscriptores y eliminarlos de esa lista.

1.3.3 – Catálogo - Patrones de comportamiento

Observer (<https://refactoring.guru/es/design-patterns/observer>)

Consecuencias (Ventajas)

- *Principio de abierto/cerrado*. introducir nuevas clases suscriptoras sin tener que cambiar el código de la notificadora (y viceversa si hay una interfaz notificadora).
- Establecer relaciones entre objetos durante el tiempo de ejecución.

1.3.3 – Catálogo - Patrones de comportamiento

Observer (<https://refactoring.guru/es/design-patterns/observer>)

Consecuencias (Desventajas)

- Los suscriptores son notificados en un orden aleatorio.